

ASSIGNMENT 2

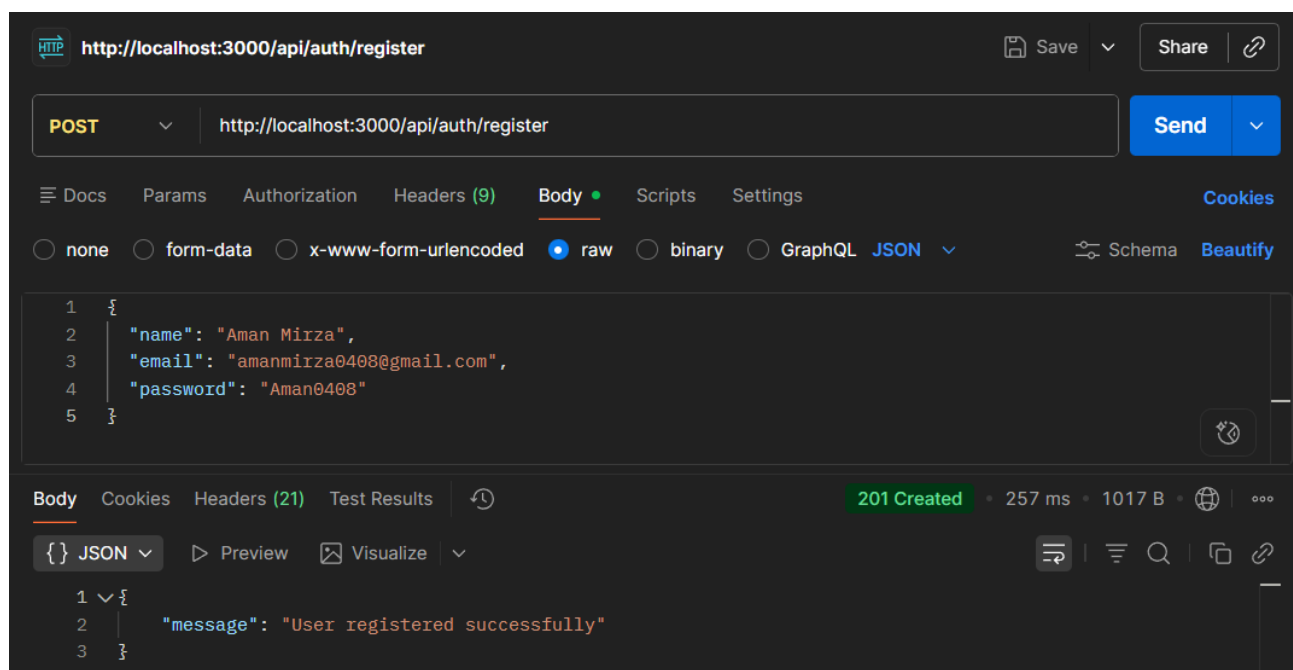
QUESTION 1

1. Blog Management REST API with Authentication (Node.js, Express, MongoDB, EJS)

Scenario: A startup wants to build the backend and a basic server-rendered interface for a personal blogging platform where registered users can sign up, log in, and manage their own blog posts. Only the author of a post (or an admin) should be able to edit or delete it. You have been asked to build this system. Build an application using Node.js, Express.js, MongoDB, Mongoose, and EJS (for rendering views) that implements the following:

1. User Authentication

- POST /api/auth/register – Register a new user (name, email, password) with the password hashed using bcrypt.



- **POST /api/auth/login** – Authenticate a user and return a JWT token.

The screenshot shows a REST client interface with the URL `http://localhost:3000/api/auth/login`. The request method is **POST**. The request body is in **raw** format (JSON) and contains the following data:

```
1 {
2   "name": "Aman Mirza",
3   "email": "amanmirza0408@gmail.com",
4   "password": "Aman0408"
5 }
```

The response status is **200 OK** with a response time of 259 ms and a size of 1.18 KB. The response body is in **JSON** format and contains the following data:

```
1 {
2   "message": "Login successful",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2YTk1NTM0OTA3MDU1Y2Y0MWUwMDRjMDMiLCJyb2xlIjoiaWoiImhhdCI6MTc4ODU3MTI1MSwIZXhwIjozNzg4MTc0ODUxZ0.-SdKoYNkus2lX1SjJ39wjn-6Cotzo92jBQelB6OKCTo"
4 }
```

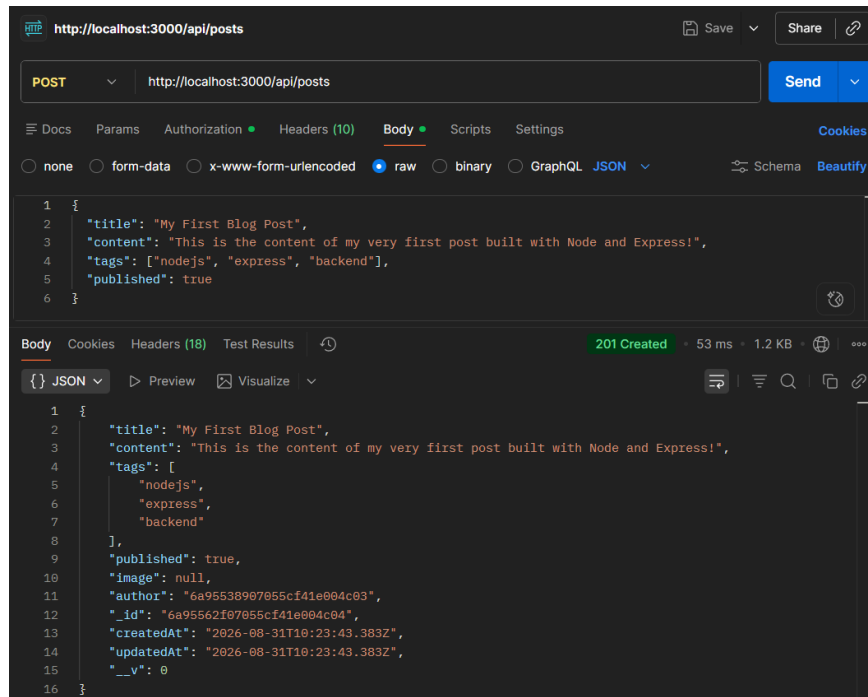
- **GET /api/auth/profile** – A protected route that returns the logged-in user's profile.

The screenshot shows a REST client interface with the URL `http://localhost:3000/api/auth/profile`. The request method is **GET**. The **Authorization** tab is active, showing the **Bearer Token** type and the token value: `IX1SjJ39wjn-6Cotzo92jBQelB6OKCTo`. The response status is **200 OK** with a response time of 23 ms and a size of 1.13 KB. The response body is in **JSON** format and contains the following data:

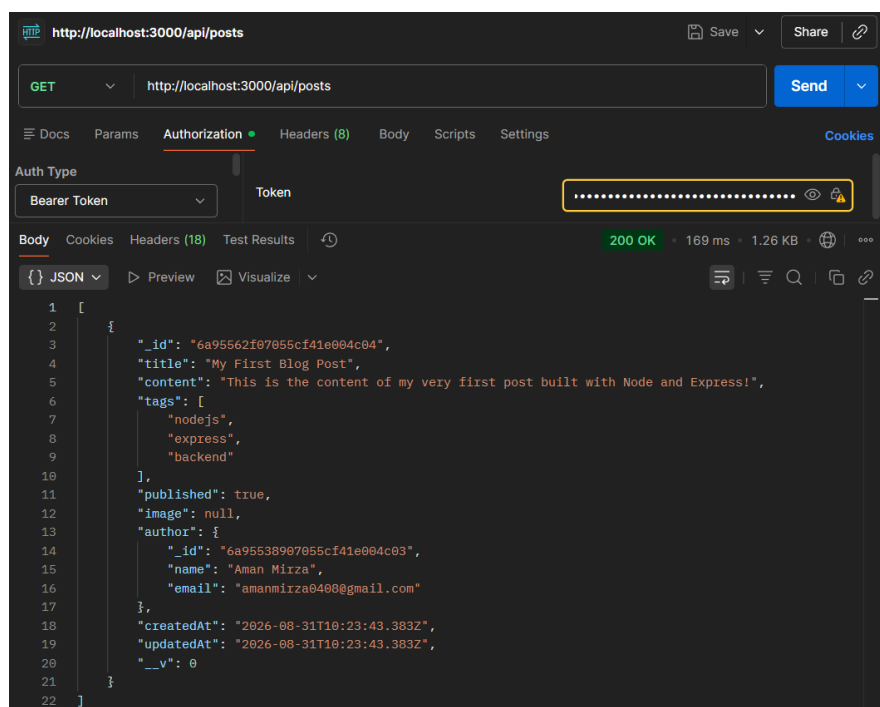
```
1 {
2   "_id": "6a95538907055cf41e004c03",
3   "name": "Aman Mirza",
4   "email": "amanmirza0408@gmail.com",
5   "role": "user",
6   "createdAt": "2026-08-31T10:12:25.724Z",
7   "updatedAt": "2026-08-31T10:12:25.724Z",
8   "__v": 0
9 }
```

2. Blog Post CRUD API (protected routes, linked to the logged-in author)

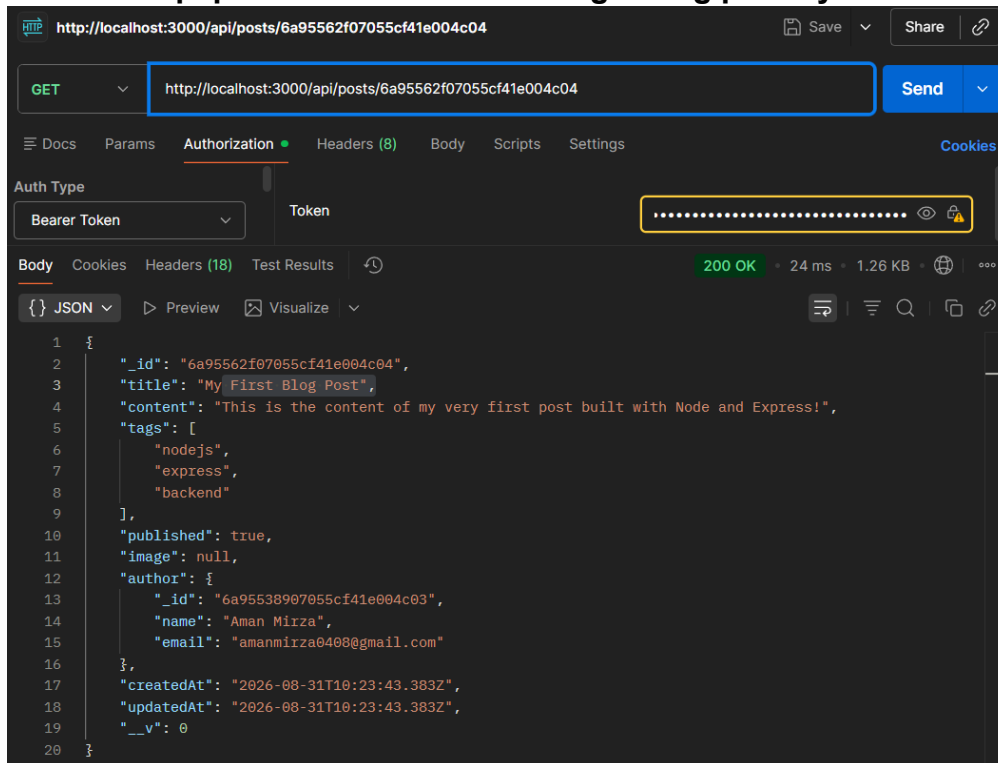
- **POST /api/posts** – Create a new blog post (title, content, tags, published status).



- **GET /api/posts** – Retrieve all blog posts belonging to the logged-in user.



- **GET /api/posts/:id – Retrieve a single blog post by ID.**

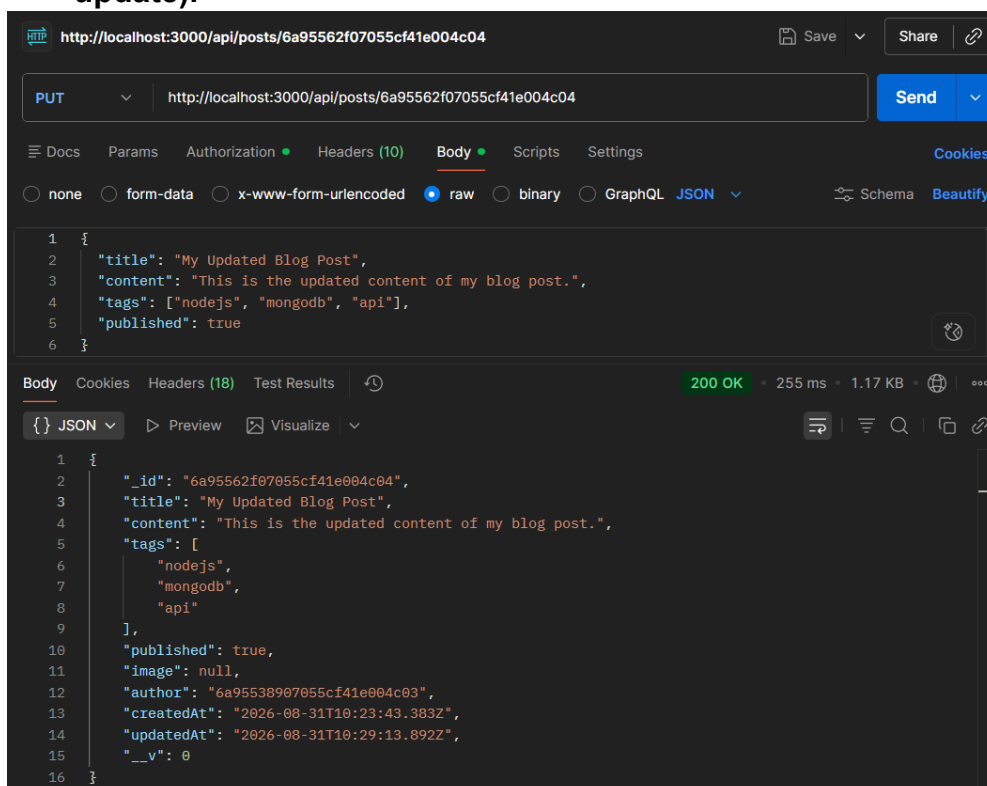


The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/posts/6a95562f07055cf41e004c04`
- Method:** GET
- Auth Type:** Bearer Token
- Token:** [Redacted]
- Status:** 200 OK
- Body:** JSON
- Response:**

```
1 {
2   "_id": "6a95562f07055cf41e004c04",
3   "title": "My First Blog Post",
4   "content": "This is the content of my very first post built with Node and Express!",
5   "tags": [
6     "nodejs",
7     "express",
8     "backend"
9   ],
10  "published": true,
11  "image": null,
12  "author": {
13    "_id": "6a95538907055cf41e004c03",
14    "name": "Aman Mirza",
15    "email": "amanmirza0408@gmail.com"
16  },
17  "createdAt": "2026-08-31T10:23:43.383Z",
18  "updatedAt": "2026-08-31T10:23:43.383Z",
19  "__v": 0
20 }
```

- **PUT /api/posts/:id – Update a blog post by ID (only the author or an admin can update).**



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/posts/6a95562f07055cf41e004c04`
- Method:** PUT
- Body Type:** raw
- Status:** 200 OK
- Body:** JSON
- Request Body:**

```
1 {
2   "title": "My Updated Blog Post",
3   "content": "This is the updated content of my blog post.",
4   "tags": ["nodejs", "mongodb", "api"],
5   "published": true
6 }
```
- Response:**

```
1 {
2   "_id": "6a95562f07055cf41e004c04",
3   "title": "My Updated Blog Post",
4   "content": "This is the updated content of my blog post.",
5   "tags": [
6     "nodejs",
7     "mongodb",
8     "api"
9   ],
10  "published": true,
11  "image": null,
12  "author": "6a95538907055cf41e004c03",
13  "createdAt": "2026-08-31T10:23:43.383Z",
14  "updatedAt": "2026-08-31T10:29:13.892Z",
15  "__v": 0
16 }
```

- **DELETE /api/posts/:id** – Delete a blog post by ID (only the author or an admin can delete).

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/posts/6a9557ce07055cf41e004c05`
- Method:** DELETE
- Body (raw):**

```

1 {
2   "title": "My Second Blog Post",
3   "content": "This is my Second blog post created using Node.js, Express and MongoDB.",
4   "tags": ["nodejs", "express", "mongodb"],
5   "published": true
6 }
```
- Status:** 200 OK, 25 ms, 950 B
- Response Body (JSON):**

```

1 {
2   "message": "Post and associated image deleted successfully"
3 }
```

- **POST /api/posts/:id/image** – Upload or replace a featured image for a blog post (multipart/form-data) using Multer.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/api/posts/6a95562f07055cf41e004c04/image`
- Method:** POST
- Body (form-data):**

Key	Value	Description
image	83-839064_lion-king-baby...	
- Status:** 200 OK, 407 ms, 983 B
- Response Body (JSON):**

```

1 {
2   "message": "Image uploaded successfully",
3   "image": "uploads/image-1788172366554-645572704.jpg"
4 }
```

3. EJS Views

- Render an EJS page (views/posts.ejs) that displays all published blog posts in a styled list.

My Updated Blog Post

By Aman Mirza



This is the updated content of my blog post.

#nodejs

#mongodb

#api

- Render an EJS page (views/login.ejs) with a login form that submits to your login route.

Welcome Back

Log In

Don't have an account? [Sign up](#)

- Render an EJS page (views/dashboard.ejs, protected) showing the logged-in user's own posts with edit/delete links, visible only after login.

Dashboard

[View Public Blog](#)

[Logout](#)

Manage Your Posts



Published

My Updated Blog Post

This is the updated content of my blog post....

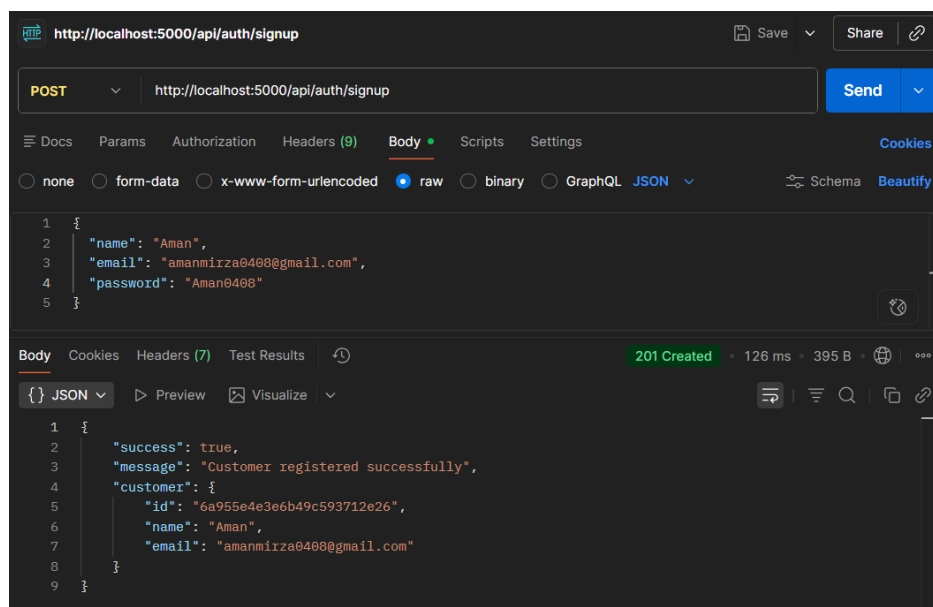
[Delete Post](#)

QUESTION 2

2. Secure Signup for Bookstore Customers

Scenario: A new customer is creating an account on the online bookstore to start placing orders for books.

Task: Write a program that hashes the customer's password using *bcrypt* before storing it in MongoDB, and write the corresponding logic to verify the password when the same customer logs in later.

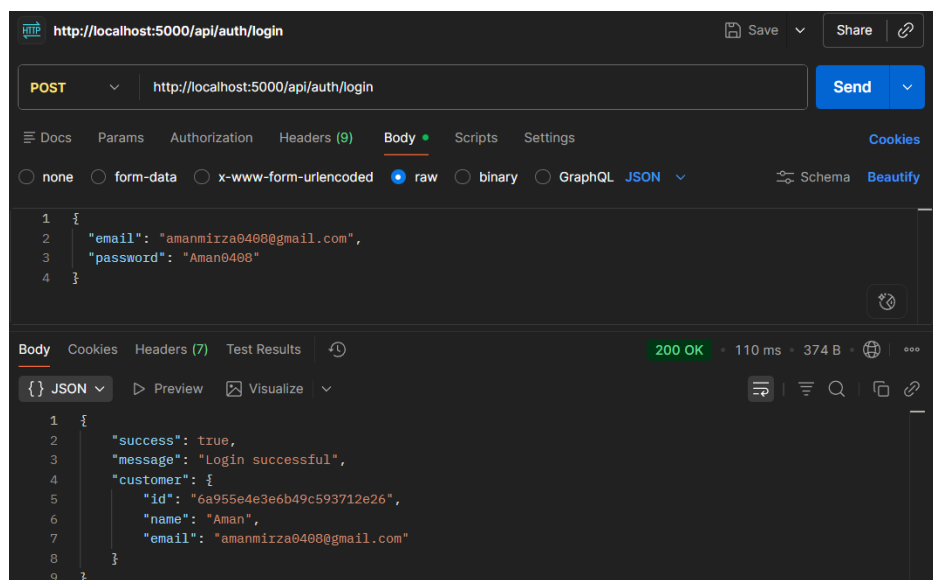


The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5000/api/auth/signup`
- Method:** `POST`
- Body (raw):**

```
1 {  
2   "name": "Aman",  
3   "email": "amanmirza0408@gmail.com",  
4   "password": "Aman0408"  
5 }
```
- Response:** `201 Created` (126 ms, 395 B)
- Body (JSON):**

```
1 {  
2   "success": true,  
3   "message": "Customer registered successfully",  
4   "customer": {  
5     "id": "6a955e4e3e6b49c593712e26",  
6     "name": "Aman",  
7     "email": "amanmirza0408@gmail.com"  
8   }  
9 }
```



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5000/api/auth/login`
- Method:** `POST`
- Body (raw):**

```
1 {  
2   "email": "amanmirza0408@gmail.com",  
3   "password": "Aman0408"  
4 }
```
- Response:** `200 OK` (110 ms, 374 B)
- Body (JSON):**

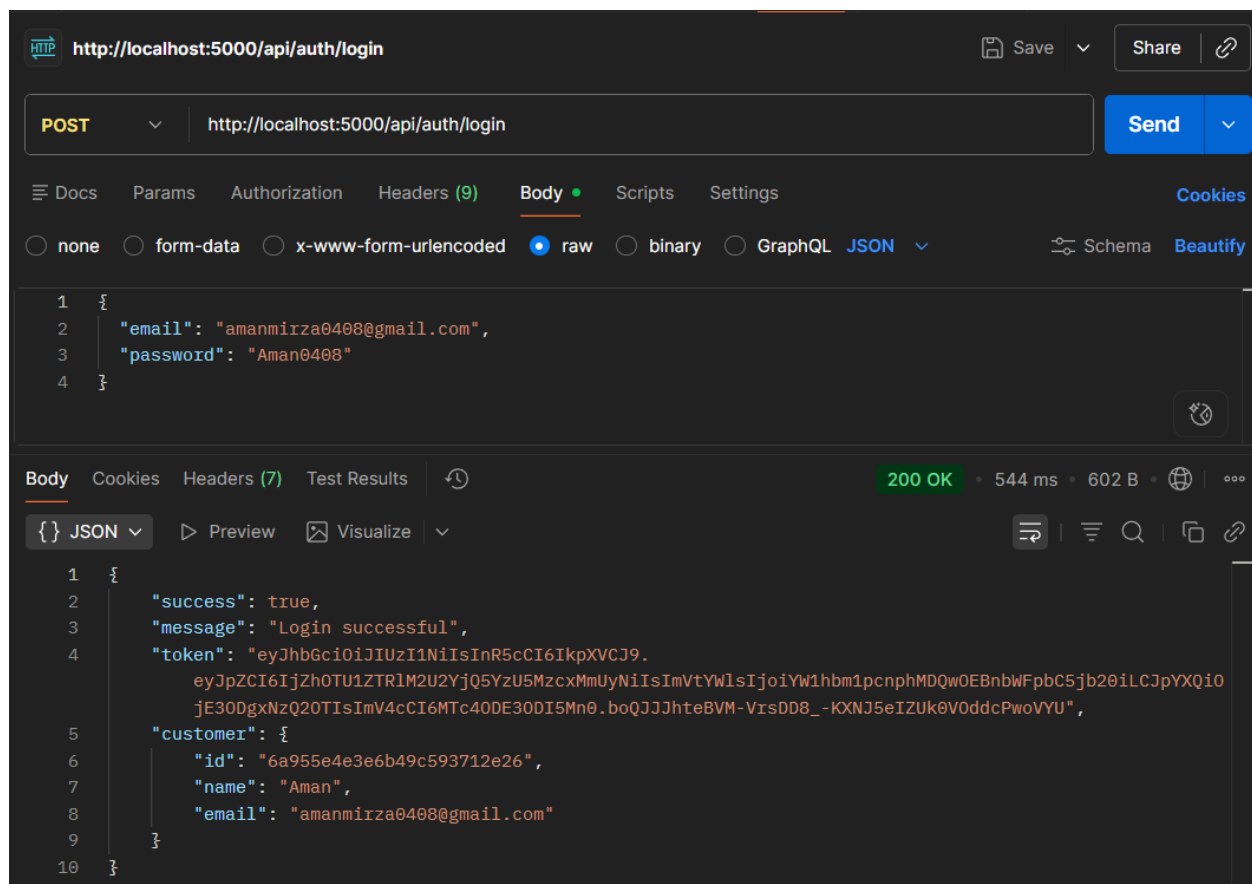
```
1 {  
2   "success": true,  
3   "message": "Login successful",  
4   "customer": {  
5     "id": "6a955e4e3e6b49c593712e26",  
6     "name": "Aman",  
7     "email": "amanmirza0408@gmail.com"  
8   }  
9 }
```

QUESTION 3

3. Issuing a Digital Membership Token on Login

Scenario: Once a customer logs in successfully, the store must issue them a token (their digital membership pass) so they don't need to log in again for every request, such as viewing their orders or checking out.

Task: Write a program to generate a JWT token on successful login, and a middleware to verify this token before allowing access to any protected route (e.g., placing an order or viewing order history).

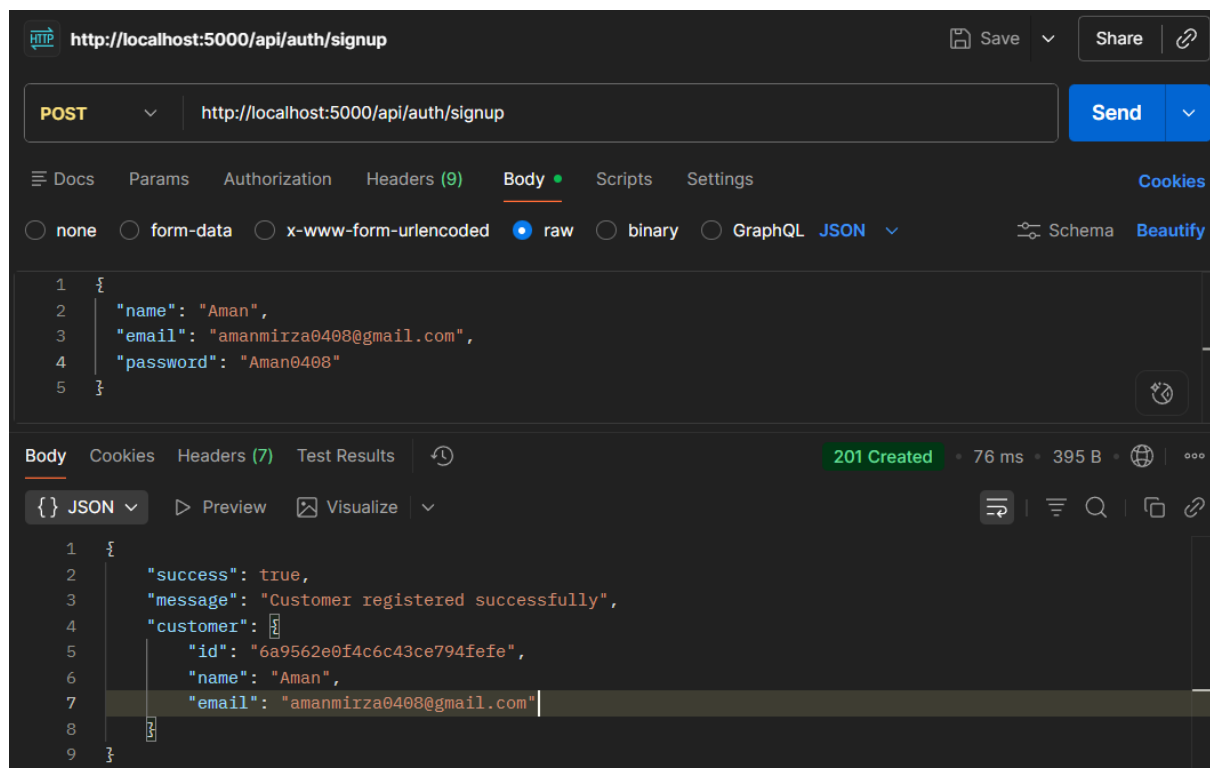


QUESTION 4

4. Defining Customer and Admin Roles

Scenario: The bookstore platform has two kinds of accounts regular Customers who browse and buy books, and Admins who manage inventory, pricing, and all customer orders.

Task: Write a Mongoose User schema with a role field ('customer' / 'admin') that defaults to 'customer', so every new signup is treated as a regular customer unless explicitly promoted to admin.

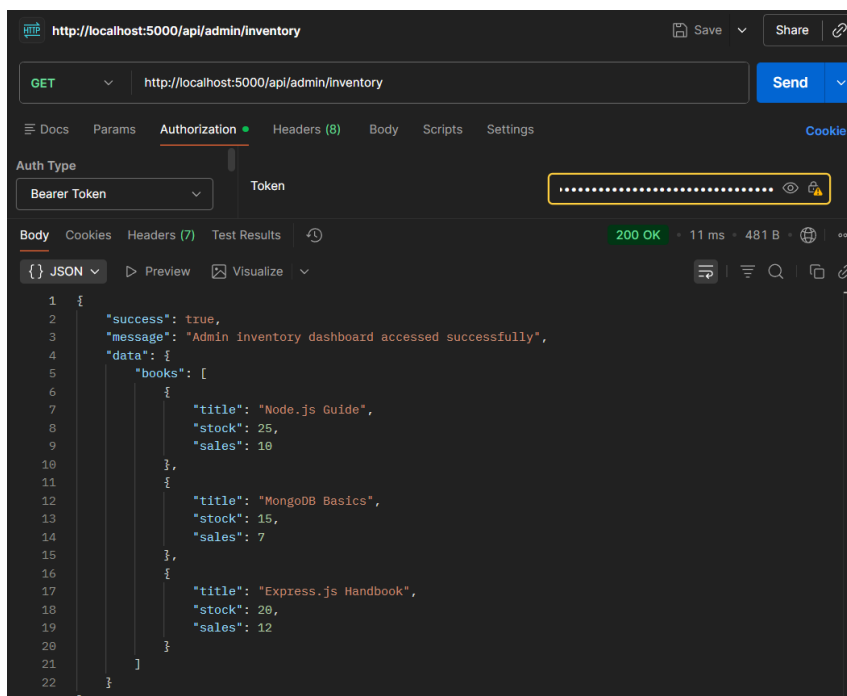
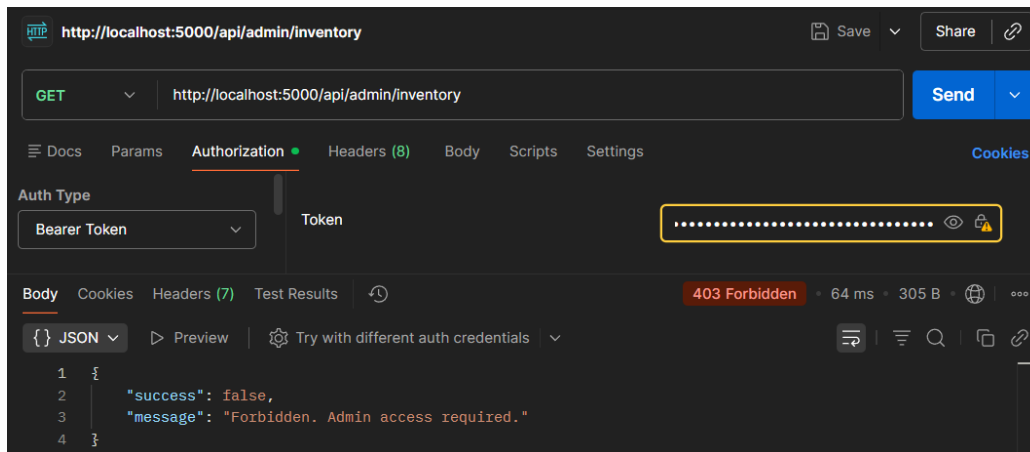


QUESTION 5

5. Admin-Only Access to the Inventory Dashboard

Scenario: The bookstore has an inventory management route (GET /api/admin/inventory) that lists stock levels and sales data for every book, meant only for store admins.

Task: Write an authorization middleware that checks the logged-in user's role and returns 403 Forbidden if the user is not an 'admin', preventing regular customers from accessing this route.



PROGRAM 6

6. Cancelling an Order — Customer or Admin Only

Scenario: A customer should be able to cancel their own order before it ships. Separately, an admin should be able to cancel any customer's order for example, when a book goes out of stock.

Task: Write a **DELETE** `/api/orders/:id` handler that allows an order to be cancelled only if the logged-in user is the customer who placed the order, or holds the admin role, and returns **403 Forbidden** otherwise.

